

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

14 - Making Decisions

Introduction

- What distinguishes a computer from a simple calculator is its ability to make decisions
- Based on input data and values created during computation, different instructions execute
- Decision making is commonly represented in programming languages using **if** statement

Conditional Branch

- Assembly language includes two decision-making instructions
 - beq, bne
 - Traditionally called conditional branches
- An instruction that requires comparison of two values and that allows for a subsequent transfer of control to a new address in the program based on the outcome of the comparison

Branch-Equal Instruction

- Instruction

`beq register1, register2, L1`

means go to instruction labeled `L1` if value in register1 equals value in register2

- Mnemonic `beq` stands for branch if equal

Branch-not-Equal Instruction

- Instruction

`bne register1, register2, L1`

means go to instruction labeled `L1` if value in register1 does not equals value in register2

- Mnemonic `beq` stands for branch if not equal

Unconditional Branch

- This instruction says that the processor always follows branch
 - Without any condition
- To distinguish between conditional and unconditional branches, MIPS name for this type of instruction is *jump*, (*j*)
- Label Exit is defined below

j Exit # go to Exit

Example

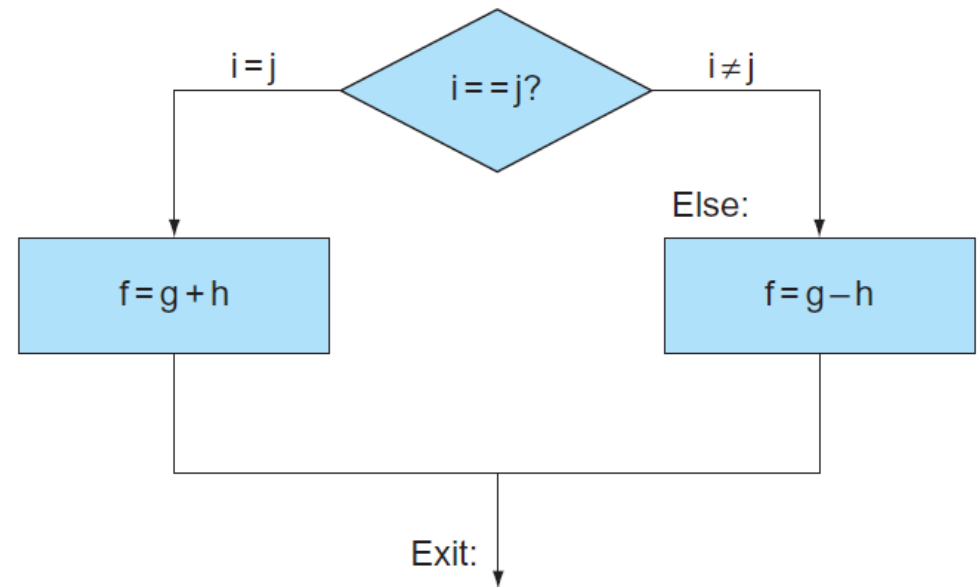
- In the following code, f, g, h, i, and j are variables. If variables f through j correspond to registers \$s0 through \$s4, what is the compiled MIPS code?

```
if (i == j) f = g + h;  
else f = g - h;
```

Example

- In the following code, f , g , h , i , and j are variables. If variables f through j correspond to registers $\$s0$ through $\$s4$, what is the compiled MIPS code?

```
if (i == j) f = g + h;  
else f = g - h;
```



Example (cont.)

- Code will be more efficient if we test for opposite condition to branch over code that performs *then* part of *if*, so use `bne`
`bne $s3,$s4,Else` # go to Else if $i \neq j$
- Next assignment statement performs single operation
`add $s0,$s1,$s2` # $f = g + h$ (skipped if $i \neq j$)

Example (cont.)

- Assignment statement in the *else* portion of the *if* statement can be compiled into single instruction
 - We need to append label **Else** to this instruction
 - We also show label **Exit** that is after this instruction

Else: `sub $s0,$s1,$s2 # f=g-h (skipped if i=j)`

Exit:

Example (cont.)

- Java code

```
if (i == j) f = g + h;  
else f = g - h;
```

- Compiled MIPS code

```
bne $s3,$s4,Else  
add $s0,$s1,$s2  
Else: sub $s0,$s1,$s2  
Exit:
```

